

Rapport — Application Flutter de Ventes Privées

Table des matières

1. Architecture choisie	1
1.1. Vue d'ensemble	1
1.2. Backend (Python / Flask)	1
1.3. Frontend (Flutter / Dart)	2
2. Difficultés rencontrées	2
2.1. 1. Communication réseau entre le serveur et l'émulateur	3
2.2. 2. Gestion des états asynchrones	3
2.3. 3. Persistance des favoris	3
2.4. 4. Filtrage côté serveur	3
3. Limites de sécurité identifiées	3

1. Architecture choisie

1.1. Vue d'ensemble

L'application suit une architecture **client-serveur** classique : deux parties indépendantes communiquent via une API REST en HTTP.

- Un **backend Python (Flask)** exposant des endpoints et lisant les données depuis des fichiers JSON.
- Un **frontend Flutter (Dart)** consommant l'API et gérant l'état local avec le pattern **Provider**.

Couche	Technologie
Backend	Python 3 + Flask 3.1
Frontend	Flutter 3.13+ (Dart 3)
État	Provider (ChangeNotifier)
Persistance session	<code>shared_preferences</code>
HTTP	paquet <code>http</code>
Stockage	Fichiers JSON locaux

1.2. Backend (Python / Flask)

Le serveur (`serveur/serveur.py`) est un fichier unique qui :

1. Charge les fichiers `users.json` et `ventes.json` en mémoire au démarrage via `load_data()`.
2. Expose quatre endpoints REST :
 - `POST /api/login` — authentification (email + mot de passe)
 - `POST /api/register` — création de compte (bonus)
 - `GET /api/produits` — liste des produits avec filtres optionnels `?search=` et `?categorie=`
 - `GET /api/produits/<id>` — détail d'un produit
3. Renvoie les réponses au format JSON avec gestion des erreurs (400, 401, 404, 409).

Le mot de passe est volontairement **exclu** de la réponse de connexion pour ne pas l'exposer côté client. Les filtres (recherche textuelle et catégorie) sont appliqués côté serveur avec un **AND** logique.

1.3. Frontend (Flutter / Dart)

L'application Flutter est organisée en trois couches :

- `models/` — contient le modèle `User` avec les champs `id`, `email`, `nom`, `prenom`.
- `services/` — trois providers :
 - `AuthProvider` : connexion, déconnexion et persistance via `SharedPreferences`.
 - `FavoritesProvider` : gestion des favoris localement (liste d'IDs persistée en JSON dans `SharedPreferences`).
 - `api_service.dart` : constante contenant l'URL de base de l'API.
- `screens/` — quatre écrans :
 - `login_screen.dart` : formulaire de connexion avec validation et message d'erreur.
 - `register_screen.dart` : formulaire d'inscription.
 - `product_screen.dart` : grille de produits avec barre de recherche, filtres par catégorie et bouton favoris.
 - `product_detail_screen.dart` : affichage détaillé d'un produit.

1.3.1. Gestion d'état avec Provider

L'application utilise `MultiProvider` à la racine (`main.dart`) pour injecter `AuthProvider` et `FavoritesProvider`. Le `SplashScreen` initial vérifie la session au démarrage : si un utilisateur est déjà stocké dans `SharedPreferences`, la navigation se fait directement vers la liste des produits, sinon l'écran de connexion s'affiche.

2. Difficultés rencontrées

2.1. 1. Communication réseau entre le serveur et l'émulateur

Sous Android, l'émulateur ne peut pas joindre `localhost` de l'hôte. Il faut utiliser l'adresse `10.0.2.2` qui redirige vers la machine hôte. Pour un appareil réel, il faut l'IP locale (`172.16.2.126`). Une URL configurable dans `api_service.dart` a permis de résoudre le problème.

2.2. 2. Gestion des états asynchrones

L'enchaînement des appels réseau (connexion, chargement des produits) et la mise à jour de l'interface ont nécessité une synchronisation fine entre les `Future` et `notifyListeners()` dans les providers.

2.3. 3. Persistance des favoris

Les favoris sont stockés dans `SharedPreferences` sous forme de chaîne JSON (liste d'IDs). La désérialisation en `Set<int>` au chargement et la sérialisation inverse à la sauvegarde ont demandé une attention particulière pour éviter les doublons.

2.4. 4. Filtrage côté serveur

L'implémentation des filtres (recherche textuelle et catégorie) combine les deux critères avec un `AND` logique. La gestion des accents, de la casse et des espaces a nécessité des ajustements.

3. Limites de sécurité identifiées

Limite	Impact
Mots de passe en clair	Stockés sans hachage dans <code>users.json</code> . Une fuite du fichier expose tous les comptes.
Absence de token JWT	L'authentification repose uniquement sur <code>SharedPreferences</code> . Aucun jeton côté serveur ne valide l'identité après la connexion.
Pas de HTTPS	Les échanges transitent en HTTP non chiffré. Sur un réseau local non sécurisé, un tiers peut intercepter les identifiants.
CORS ouvert	Le serveur accepte toutes les origines (<code>CORS(app)</code>), acceptable en développement mais dangereux en production.
Aucune validation de mot de passe	Ni côté client ni côté serveur, il n'y a de règle de complexité ou de longueur minimale.